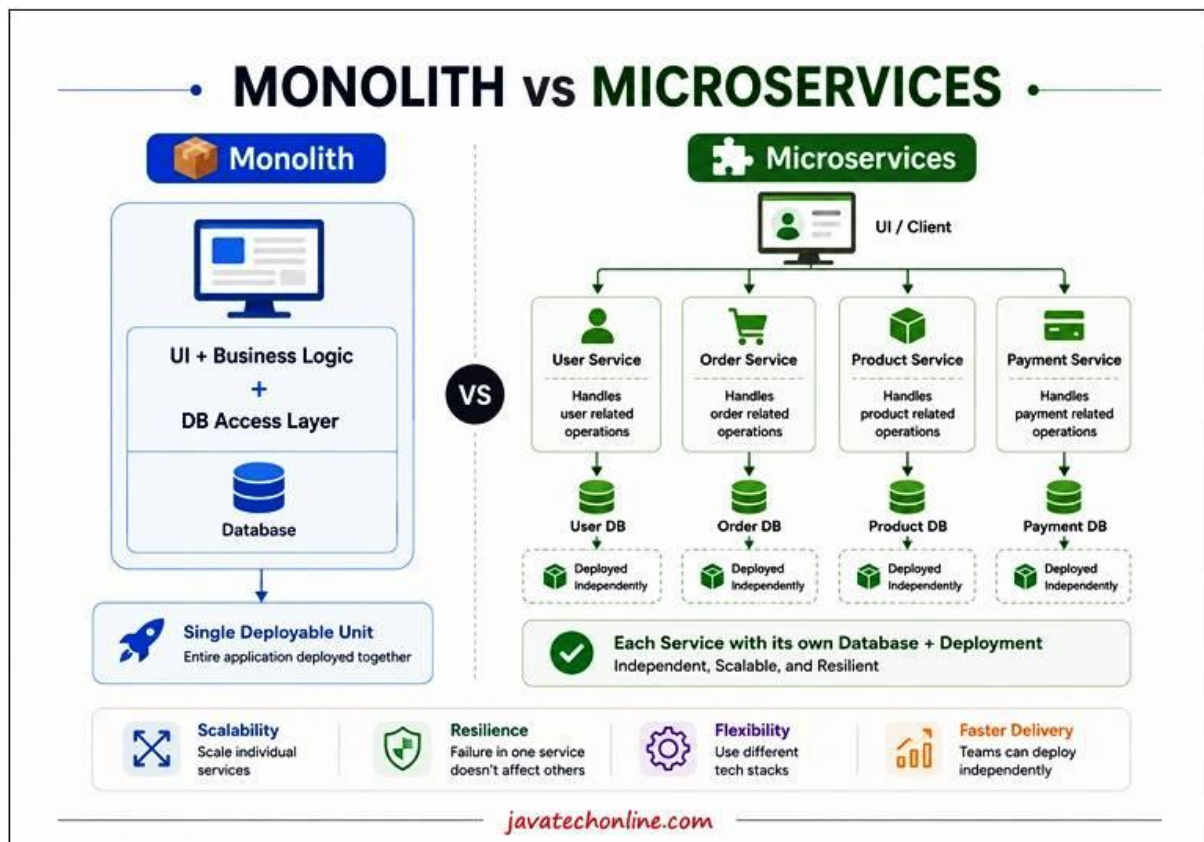


What Are Microservices?

Imagine a large e-commerce app like Amazon. Instead of building the entire application as one massive codebase (called a **monolith**), you split it into small, independently deployable services: one for Orders, one for Users, one for Payments, one for Products. Each service has its own database, runs in its own process, and communicates with others via APIs or messages. **That's microservices.**



Q1. What is the difference between Monolithic and Microservices Architecture?

Feature	Monolithic	Microservices
Structure	Single deployable unit	Multiple independent services
Scaling	Scale the whole app	Scale individual services
Technology	One tech stack	Polyglot (different stacks per service)
Deployment	Deploy all at once	Deploy each service independently
Fault isolation	One failure can crash the app	Failure isolated to one service
Team size	Works for small teams	Better for large, distributed teams

Real-World Tip: Netflix started as a monolith and migrated to microservices to handle 200+ million users. Interviewers love this example.

Q2. What are the core features of Microservices?

- **Decoupling** – Each service works independently, so changes in one don't break others

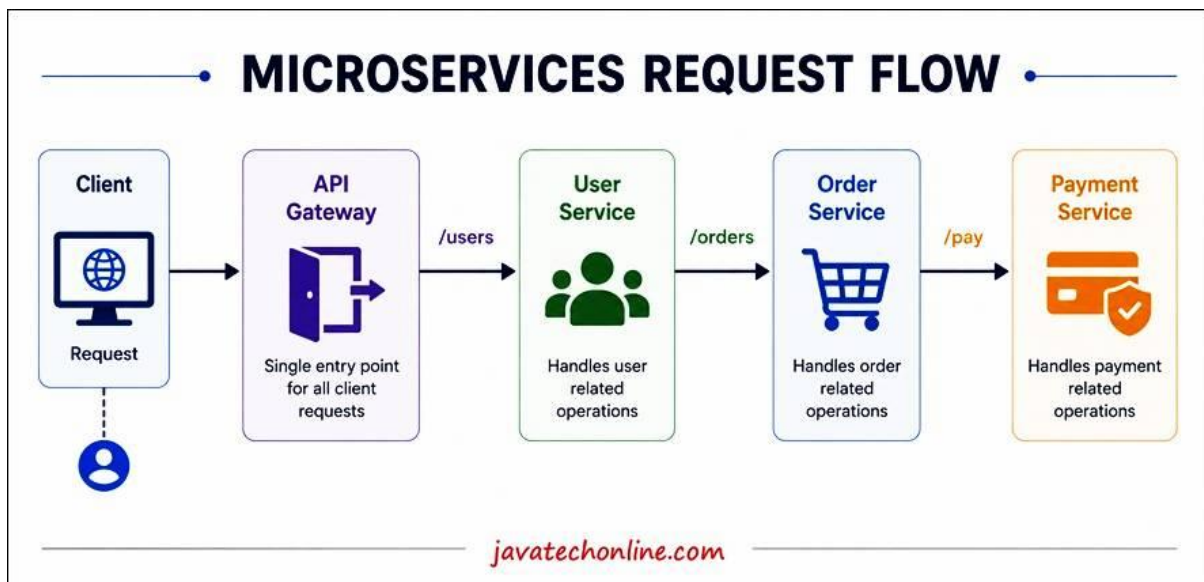
- **Single Responsibility** – Each service does one job and does it well (e.g., Notification Service only sends emails/SMS)
- **Autonomous Deployment** – You can deploy the Payment Service without touching the User Service
- **Fault Tolerance** – If the Recommendation Service goes down, the rest of the app keeps working
- **Polyglot Persistence** – Orders DB can use MySQL, while the Search Service uses Elasticsearch

Q3. What is an API Gateway, and why is it needed?

Think of the API Gateway as the **front door** of your microservices system. Clients (web, mobile) never talk directly to individual services. Instead, they hit the API Gateway, which:

- Routes the request to the right service
- Handles authentication and authorization
- Does SSL termination
- Manages rate limiting and throttling
- Aggregates responses from multiple services

In Java/Spring: Spring Cloud Gateway or Netflix Zuul are the most popular choices.



Q4. What is Service Discovery, and how does Eureka work?

In a microservices environment, services are constantly starting, stopping, and scaling. You can't hardcode IP addresses. They change dynamically.

Service Discovery solves this. Each service registers itself with a **Service Registry** (like Netflix Eureka) when it starts up. When Service A wants to talk to Service B, it asks the registry: "Where is Service B?"

Eureka flow:

1. Service starts → registers with Eureka Server (sends host, port, service name)
2. Eureka stores this info
3. Every 30 seconds, the service sends a **heartbeat** to say "I'm alive"

4. If no heartbeat for 90 seconds → Eureka removes the service
5. Another service queries Eureka → gets the address → makes the call

In Spring Boot:

// Eureka Server

@SpringBootApplication

@EnableEurekaServer

public class EurekaServerApp { ... }

// Client Service

@SpringBootApplication

@EnableEurekaClient

public class OrderServiceApp { ... }

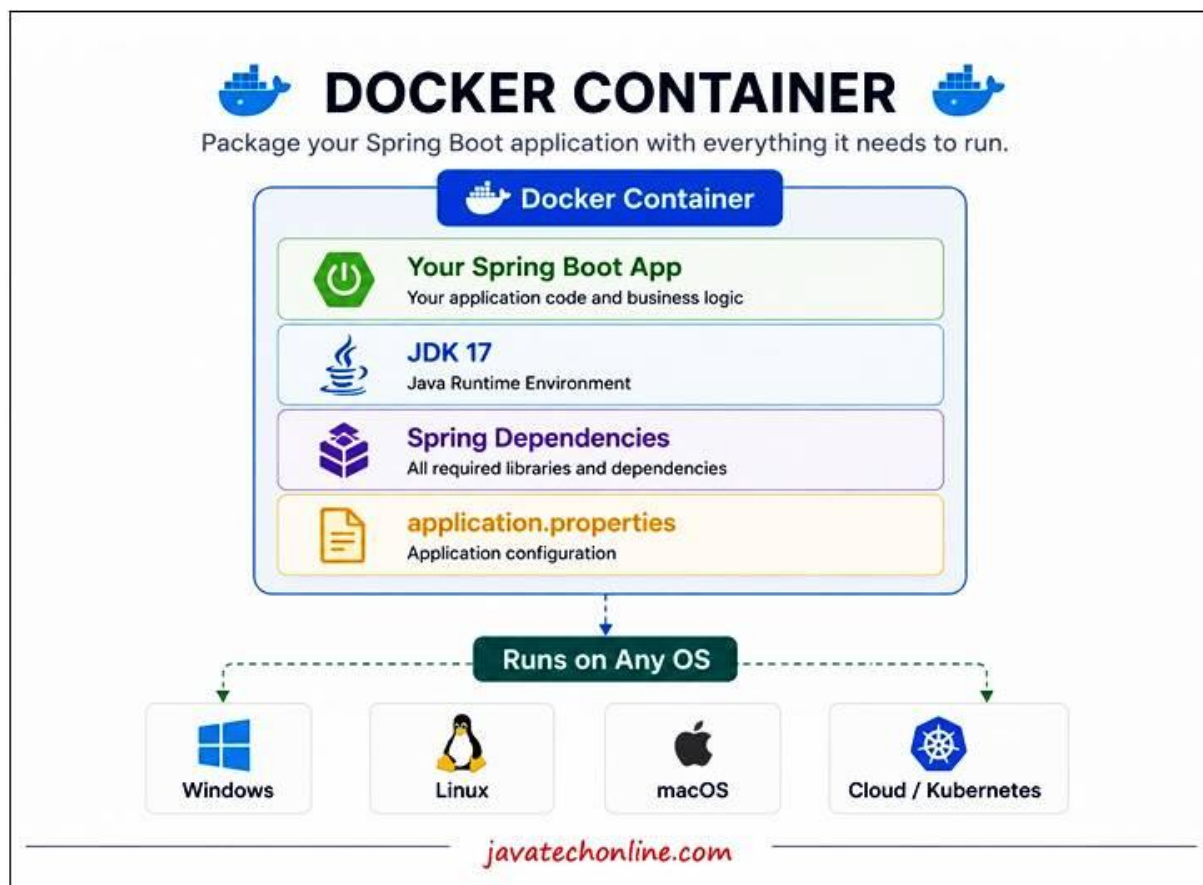
Q5. What is the difference between REST and gRPC communication in microservices?

	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Format	JSON (human-readable)	Binary Protobuf (compact, fast)
Speed	Slower	Faster (up to 10x)
Language support	Universal	Multi-language via codegen
Use case	Public APIs, browser clients	Internal service-to-service calls
Spring support	Built-in (Spring Web)	Spring gRPC (newer)

Q6. What is a Docker Container, and why does every microservice need one?

A Docker container packages your Spring Boot service **along with all its dependencies** (JDK, libraries, config) into a portable, self-contained unit. This means:

- Runs the same on your laptop, CI server, and AWS production
- Services don't interfere with each other (isolated environments)
- Easy to scale: spin up 5 more containers of the Order Service in seconds



Q7. What is Spring Cloud, and which problems does it solve?

Spring Cloud is a collection of tools built on top of Spring Boot to address common microservices challenges:

Problem	Spring Cloud Solution
Service discovery	Spring Cloud Netflix Eureka
Config management	Spring Cloud Config Server
Load balancing	Spring Cloud LoadBalancer
API routing	Spring Cloud Gateway
Circuit breaking	Spring Cloud + Resilience4j
Distributed tracing	Spring Cloud Sleuth + Zipkin

Q8. What is client-side vs. server-side load balancing?

Server-side load balancing: A dedicated load balancer (like AWS ELB or Nginx) sits between the client and services. The client only talks to the load balancer; it doesn't know about individual instances.

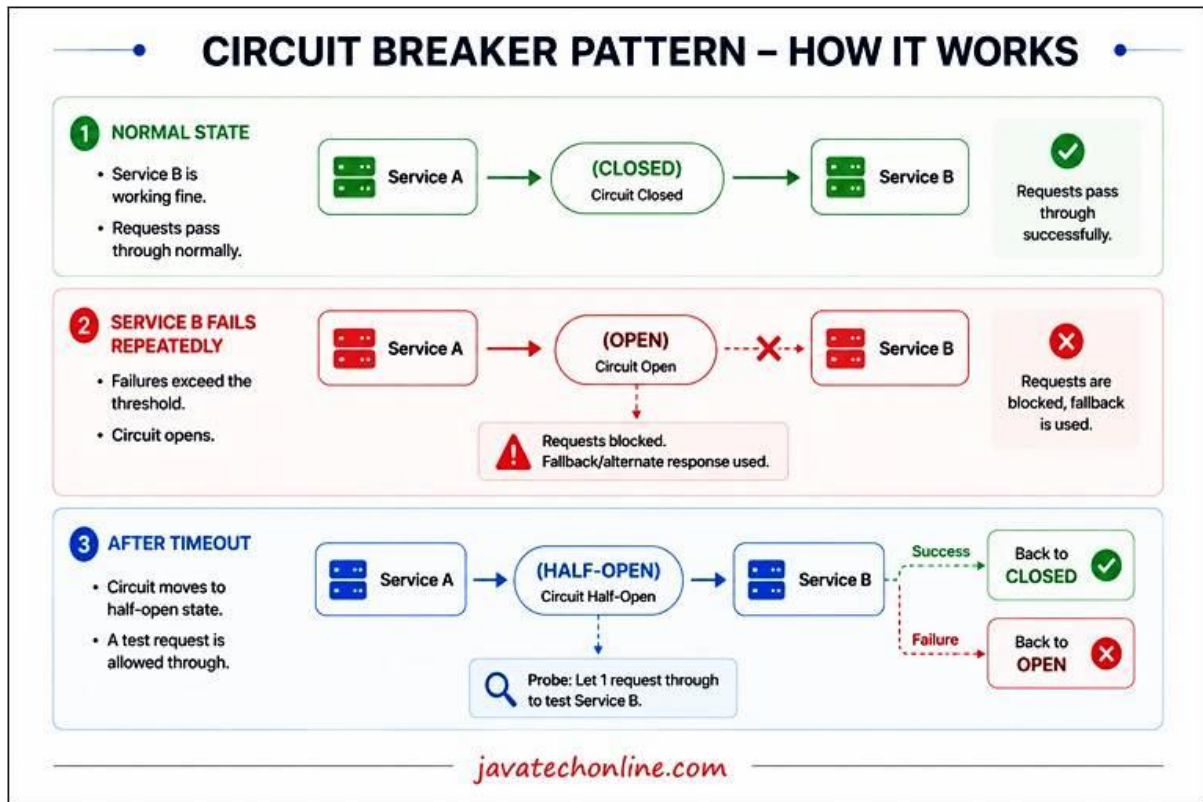
Client-side load balancing: The client itself decides which instance to call. It fetches the list of instances from Eureka and picks one (using Round Robin, Random, etc.). Spring Cloud LoadBalancer works this way.

Interview insight: Ribbon (now deprecated) was Netflix's client-side LB. Spring Cloud LoadBalancer replaced it. Know this difference. It comes up often.

Q9. What is the Circuit Breaker pattern? Explain with states.

Imagine Service A calls Service B. Service B is down or very slow. Without protection, Service A keeps waiting, eventually its own thread pool fills up and it crashes too. This is called **cascading failure**.

The **Circuit Breaker pattern** prevents this. It works like an electrical circuit breaker:



In Spring Boot with Resilience4j:

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
```

```
public String makePayment(Order order) {  
    return paymentClient.pay(order);  
}
```

```
public String fallback(Order order, Exception ex) {  
    return "Payment temporarily unavailable. Please retry.";  
}
```

Q10. How do microservices communicate with each other?

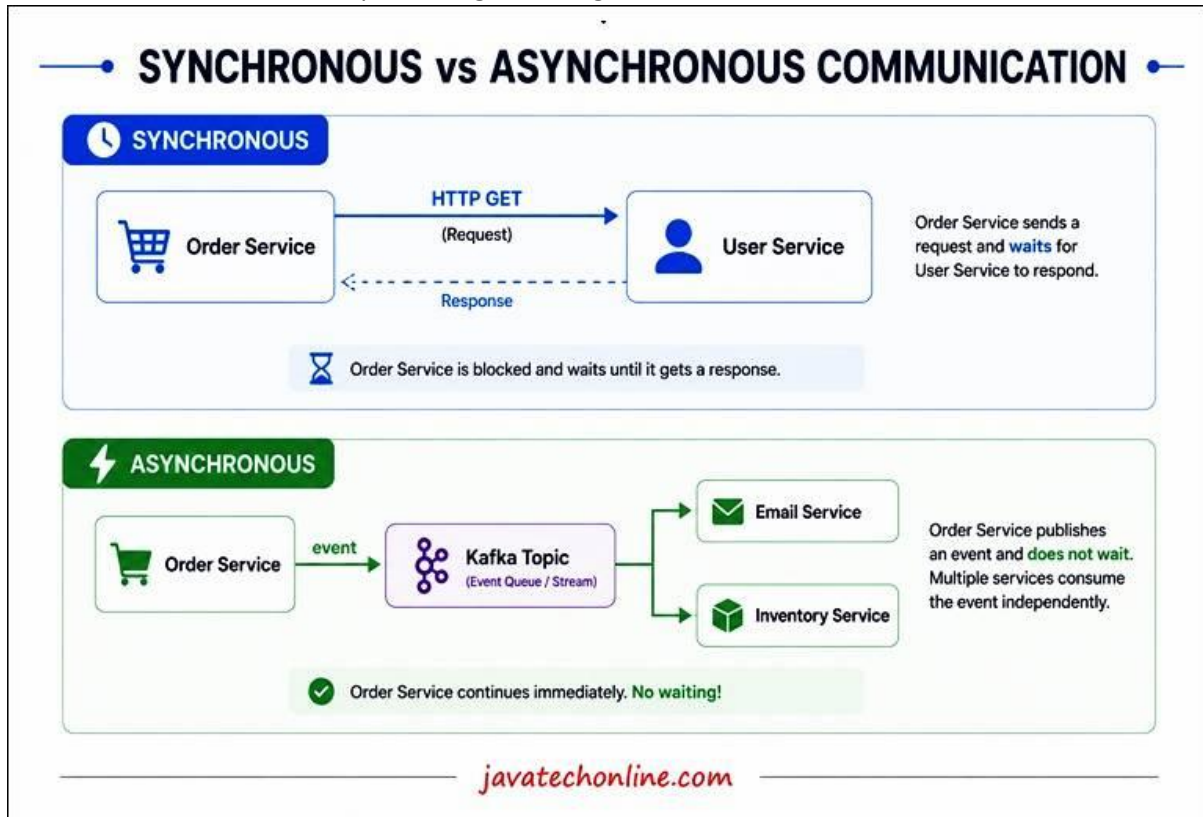
There are two styles:

1. Synchronous (request-response):

- Service A calls Service B and waits for the response
- Tools: RestTemplate, OpenFeign, WebClient
- Good for: real-time queries (e.g., "Is this user valid?")

2. Asynchronous (event-driven):

- Service A publishes an event to a Message Broker; Service B picks it up later
- Tools: Apache Kafka, RabbitMQ, Active MQ etc.
- Good for: notifications, order processing, audit logs



Q11. What is an OpenFeign Client, and why is it preferred over RestTemplate?

RestTemplate is the traditional way to call other services. It works, but you write a lot of boilerplate:

```
ResponseEntity response = restTemplate.getForEntity(  
    "http://user-service/users/" + id, User.class);
```

OpenFeign turns an HTTP call into a simple Java interface:

```
@FeignClient(name = "user-service")  
  
public interface UserClient {  
    @GetMapping("/users/{id}")  
    User getUserById(@PathVariable Long id);  
}
```

Feign handles service discovery (via Eureka), load balancing, retry, and error decoding, all configured via annotations. It's cleaner, testable, and the preferred choice in modern Spring Boot apps.

Q12. What is the Saga Pattern? How do you handle distributed transactions?

In a monolith, you use a single database transaction. In microservices, each service has its own DB, there's no shared transaction. So what happens when:

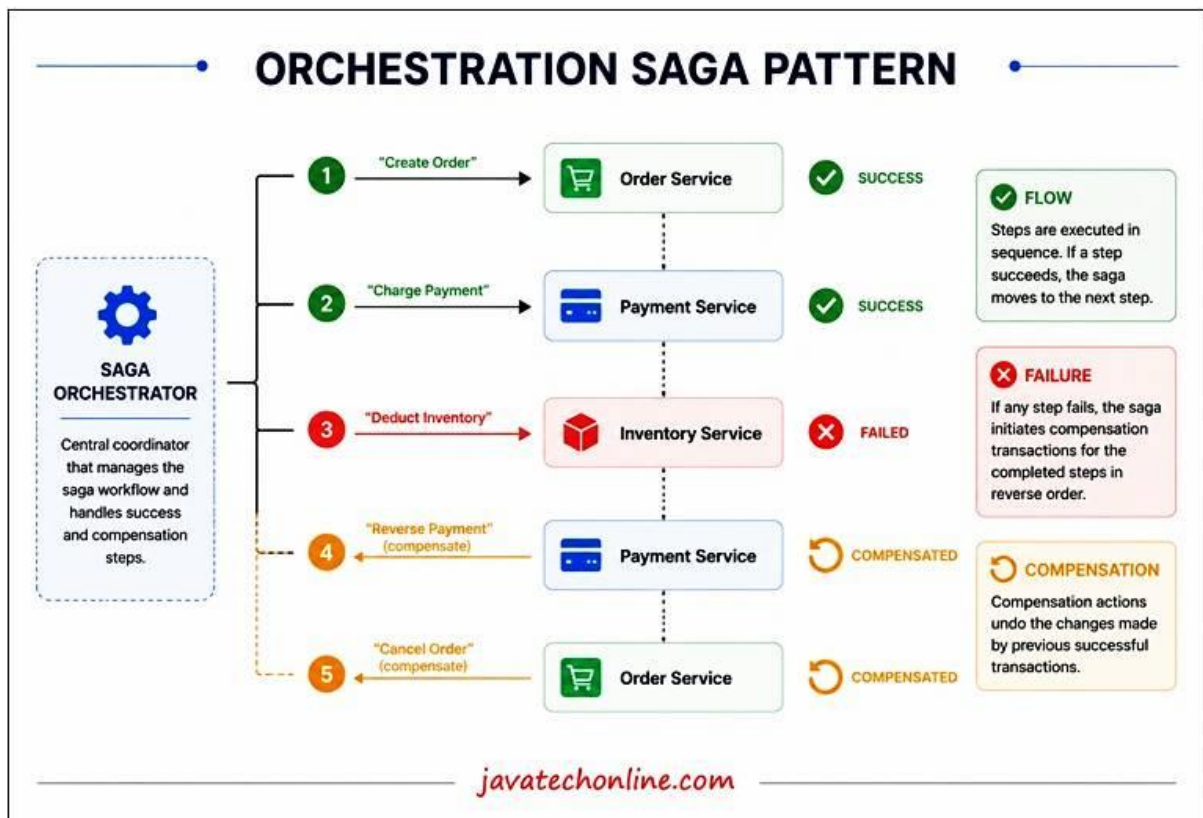
1. Order Service creates an order

2. Payment Service charges the user
3. Inventory Service deducts stock

You need to **undo** the previous steps. This is what the **Saga Pattern** handles.

Two types:

- **Choreography Saga:** Each service listens to events and reacts. No central coordinator. Simple but hard to debug.
- **Orchestration Saga:** A dedicated **Saga Orchestrator** (e.g., built with Camunda or custom code) tells each service what to do and handles compensating transactions on failure.



Q13. What is an Idempotent API, and why does it matter in microservices?

An API is **idempotent** if calling it multiple times produces the same result as calling it once.

Why it matters: In distributed systems, a request might be sent twice (due to network retries, timeouts). If your "charge payment" API isn't idempotent, the customer could be charged twice.

How to implement: Use a unique idempotency-key header. Before processing, check if this key was already processed. If yes, return the cached response.

// Pseudo code

```
if (idempotencyRepo.exists(key)) {
    return cachedResponse(key);
}

// process request
idempotencyRepo.save(key, response);
return response;
```

Q14. What is Distributed Tracing, and which tools are used?

When a request passes through 5 microservices, and something goes wrong, how do you find where it broke?

Distributed Tracing assigns a unique **Trace ID** to every incoming request. This ID travels with the request across all services. Each service logs its work with this ID, so you can see the entire journey in one place.

Tools:

- **Spring Cloud Sleuth** – Auto-injects Trace ID and Span ID into logs
- **Zipkin** – Visualizes traces in a timeline UI
- **Jaeger** – OpenTelemetry-based tracer, popular in Kubernetes setups

Q15. What is Bounded Context in Domain-Driven Design (DDD)?

A **Bounded Context** is a logical boundary within which a particular domain model is valid and consistent.

Think of the word “Account” in Banking, it means a bank account. In HR, it means a user login. These are two different contexts. DDD says: define clear boundaries so “Account” in one service doesn’t bleed into another.

Each microservice should ideally map to one Bounded Context. This is what keeps services truly independent.

Advanced Level (5–10 Years Experience)

At this level, expect deep-dive architecture and hands-on design questions.

Q16. How would you migrate a Monolith to Microservices?

This is a common senior-level question. The answer is **don’t do a big-bang rewrite**. Use the **Strangler Fig Pattern**:

1. Identify the domain that needs the most scaling (e.g., User Authentication)
2. Extract it as a microservice while keeping the monolith running
3. Route traffic to the new service via the API Gateway
4. Gradually extract more domains, strangling the monolith piece by piece

Phase 1: [Monolith] handles all traffic

Phase 2: [Monolith] + [User Service] — Gateway routes /auth to new service

Phase 3: [Monolith] + [User Service] + [Order Service]

...

Phase N: All domains extracted — Monolith retired

Q17. How do you secure microservices with JWT and OAuth2?

Security in microservices is applied at the **API Gateway level**, not per service (though per-service validation is a good defense-in-depth practice).

Flow with JWT + OAuth2:

1. User logs in → Auth Server issues JWT token
2. Client sends JWT in every request: Authorization: Bearer
3. API Gateway validates the JWT signature
4. Gateway strips/passes token to downstream services
5. Each service can optionally verify the token claims

In Spring Boot:

- Use spring-boot-starter-oauth2-resource-server

- Configure `spring.security.oauth2.resourceserver.jwt.issuer-uri`
- Token validation happens automatically

Q18. What is the difference between Orchestration and Choreography in microservices?

	Orchestration	Choreography
Control	Central orchestrator controls flow	Each service reacts to events
Coupling	Higher (services depend on orchestrator)	Lower (services only emit/listen to events)
Visibility	Easy to see the full flow	Harder to trace (events everywhere)
Failure handling	Orchestrator manages compensations	Each service handles its own rollback
Use case	Complex workflows (order processing)	Simple, scalable event flows (notifications)

Q19. How would you handle service-to-service security (Zero Trust)?

In a Zero Trust model, every service must authenticate itself, even internal services. Common approaches:

- **Mutual TLS (mTLS):** Both sides present certificates. Popular in service meshes (Istio, Linkerd)
- **JWT-based service tokens:** Each service gets a token from the Auth Server and passes it in internal calls
- **API Keys:** Simpler, but less secure used for internal tooling, not production-grade systems

In Kubernetes with Istio: mTLS is enabled by default between pods. This is the industry standard for 2025.

Q20. What is the Event Sourcing pattern?

Instead of storing the **current state** of an object, you store a log of every event that led to that state.

Traditional DB:

Order table: { id: 1, status: "SHIPPED", amount: 500 }

Event Sourced:

events:

```
[
  { type: "ORDER_CREATED", amount: 500 },
  { type: "PAYMENT_RECEIVED", amount: 500 },
  { type: "ORDER_SHIPPED", trackingId: "TRK123" }
]
```

Current state = replay of all events

Benefits: Full audit trail, time-travel queries, easy event replay for new subscribers.

Tools: Apache Kafka (as event store), Axon Framework (Java).

Q21. How do you implement CQRS in a microservices context?

CQRS (Command Query Responsibility Segregation) separates write operations (Commands) from read operations (Queries).

- **Write side:** Handles business logic, writes to a normalized DB, publishes events
- **Read side:** Listens to events, builds denormalized read models optimized for queries

This is powerful in microservices because:

- You can scale read replicas independently
- Read models can be in Redis, Elasticsearch, or any format optimized for the query
- Write and read paths are independently deployable

Client —[Create Order]—→ Command Handler → SQL DB → publishes event

↓

Client —[Get Order]—→ Query Handler ← Read Model (updated by event)

Quick-Fire Questions (Good for Last Minute Revision)

Question	Short Answer
What is the default port of Eureka Server?	8761
What annotation enables a Eureka Client?	@EnableEurekaClient
What is a Feign Client?	Declarative HTTP client for inter-service calls
Difference between PUT and PATCH?	PUT replaces the resource; PATCH partially updates
What is Spring Boot Actuator?	Exposes health, metrics, and env endpoints for monitoring
What is Resilience4j?	Modern fault-tolerance library for Spring Boot (replaced Hystrix)
What is Kafka vs. RabbitMQ?	Kafka: high-throughput log streaming; RabbitMQ: traditional message broker
What is @FeignClient?	Annotation to define a declarative REST client in Spring Cloud
How is config managed across services?	Spring Cloud Config Server (Git-backed central config)
What is Blue-Green Deployment?	Two identical environments; switch traffic to “green” after deployment

Advanced Questions & Answers

Architecture & Design Patterns

Q22. What is the Sidecar Pattern in microservices?

A sidecar is a helper container deployed alongside your main service container (in the same pod in Kubernetes). It handles cross-cutting concerns like logging, monitoring, config sync, and security without touching the main app's code. [Istio](#)'s Envoy proxy is a classic sidecar example.

Q23. What is the Bulkhead Pattern?

Named after ship compartments that prevent flooding from spreading, the Bulkhead pattern isolates service resources (thread pools, connection pools) so one slow downstream service can't consume all resources and bring down unrelated features. Resilience4j has native Bulkhead support.

Q24. What is Canary Deployment, and how does it reduce risk?

Instead of deploying a new version to all servers at once, you release it to a small percentage of users (say, 5%) first: the "canaries." You monitor error rates and latency. If stable, gradually roll out to 100%. If not, roll back only the canary. Popular in Kubernetes with tools like Argo Rollouts or Istio traffic splitting.

Q25. What is Chaos Engineering, and why do teams practice it?

[Chaos Engineering](#) is the discipline of intentionally injecting failures (kill a pod, throttle network, spike CPU) into a production-like system to find weaknesses before real outages do. Netflix's Chaos Monkey popularized this. In Java microservices, Chaos Toolkit and Litmus (Kubernetes-native) are common tools.

Data & Consistency

Q26. How do you handle data consistency across microservices without distributed transactions?

Three key strategies:

- **Eventual Consistency:** Accept that data won't be instantly consistent; use events to sync over time
- **Saga Pattern:** Chain of local transactions with compensating rollbacks
- **Outbox Pattern:** Write to a local "outbox" DB table and a separate process publishes the event to Kafka, guaranteeing at-least-once delivery

Q27. What is Database Sharding, and when do you use it in microservices?

Sharding splits a database horizontally e.g., users with IDs 1–1M go to Shard A, 1M–2M to Shard B. Each microservice can shard its own database independently. Use it when a single service's DB becomes a bottleneck at scale (e.g., a high-write Order Service).

Q28. What is the Outbox Pattern, and how does it solve dual-write problems?

The dual-write problem: You save to DB and publish to Kafka. If Kafka publish fails, your DB has data but the event is lost. The Outbox Pattern saves both the main record and the event in the same DB transaction to an "outbox" table. A separate Message Relay process reads the outbox and publishes to Kafka. Atomicity guaranteed.

Observability & Operations

Q29. What are the three pillars of observability in microservices?

- **Logs:** Structured logs (JSON) with Trace ID, correlated across services (ELK Stack: Elasticsearch, Logstash, Kibana)
- **Metrics:** Counters, histograms, gauges (Prometheus + Grafana dashboards)
- **Traces:** End-to-end request journeys across services (Jaeger, Zipkin)

Q30. How do you implement centralized logging in a Java microservices system?

Each Spring Boot service writes structured logs → shipped to **Logstash** via Filebeat → stored in **Elasticsearch** → visualized in **Kibana**. Spring Cloud Sleuth injects traceId and spanId automatically, so every log line is searchable by request ID..

Q31. What is a Service Mesh, and when do you need one?

A service mesh (Istio, Linkerd) handles all service-to-service communication transparently: mTLS, retries, circuit breaking, observability without any application code changes. You need it when you have 20+ services and managing this in each service individually becomes unmanageable.

Deployment & Scaling**Q32. What is the difference between Horizontal and Vertical Scaling in microservices?**

	Vertical Scaling	Horizontal Scaling
How	Add more CPU/RAM to one instance	Add more instances of the service
Limit	Hardware ceiling	Near-unlimited (with Kubernetes HPA)
Downtime	Yes (restart)	No
Microservices fit	Poor	Excellent

Q33. What is Blue-Green Deployment vs. Rolling Deployment?

- **Blue-Green:** Two full environments (Blue = live, Green = new version). Switch all traffic at once. Instant rollback by switching back. Expensive (double infrastructure).
- **Rolling:** Gradually replace old instances with new ones, a few at a time. Cheaper but slower rollback.

Q34. How does Kubernetes Horizontal Pod Autoscaler (HPA) work with microservices?

HPA monitors a metric (CPU usage, custom metric like requests/sec) and automatically scales the number of pods up or down. For example: if Order Service CPU > 70%, HPA spins up 2 more pods. Combined with a Service Mesh for traffic splitting, this gives you elastic scalability with zero manual intervention.

Security (Advanced)**Q35. What is the difference between Authentication and Authorization in microservices?**

- **Authentication** = Who are you? (JWT token, OAuth2 login)
- **Authorization** = What can you do? (RBAC roles, Spring Security @[PreAuthorize](#))

In microservices, Authentication is typically done at the API Gateway. Authorization can be centralized (at Gateway) or decentralized (each service checks roles from the JWT claim).

Q36. How do you rotate secrets and manage sensitive config in microservices?

Never hardcode secrets. Use: HashiCorp Vault for dynamic secret injection, AWS Secrets Manager or Azure Key Vault for cloud-native setups, and Spring Cloud Config + Vault integration for Spring Boot services. Secrets are fetched at startup or on-demand, with automatic rotation policies.